

CALLWAVE PLATFORM - UPDATED TECHNICAL DESIGN DOCUMENT

For: 400k Users in Cameroon | **Concurrent:** 1,000+ | **Date:** Sept 2026 | **Status:** PRODUCTION-READY

1. EXECUTIVE SUMMARY

Platform: 400,000 users in Cameroon, 1,000+ concurrent video calls

Technology Stack:

- Node.js + TypeScript (Backend)
- REST APIs + RabbitMQ (Async jobs)
- Mediasoup SFU (Media forwarding)
- PostgreSQL + Redis (Data layer)
- Coturn (NAT traversal)

Deployment: Docker Compose (single server) → Kubernetes (multi-server)

Architecture: Microservices (7 independent services)

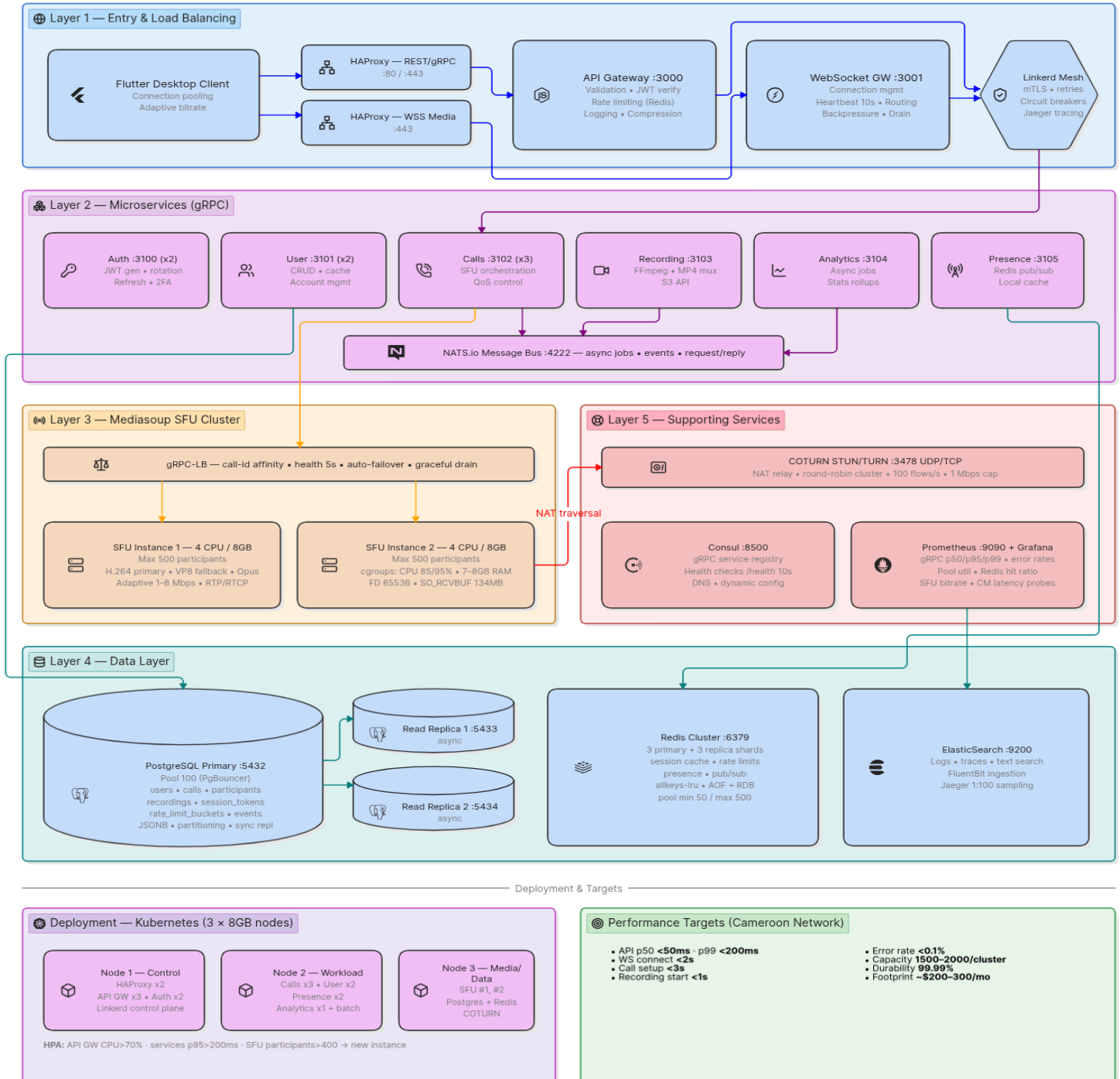
Key Performance Targets:

- API latency (p50): <100ms
 - API latency (p99): <300ms
 - Call setup time: <3 seconds
 - Call success rate: >98%
 - Error rate: <0.1%
 - Server uptime: 99.5%
-

2. ARCHITECTURE OVERVIEW

2.1 System Layers

Callwave — Optimized Architecture



2.2 Service Breakdown

Service	Port	Purpose	Technology
API Gateway	3000	REST entry point, request validation	Express + TypeScript
WebSocket GW	3001	Real-time signaling, WebRTC negotiation	Express WS
Auth Service	3100	JWT generation, login, token refresh	Express + bcryptjs
User Service	3101	User profile, contacts, search	Express + Postgres
Calls Service	3102	Call creation, routing to SFU, lifecycle	Express + Mediasoup
Recording Service	3103	FFmpeg encoding, MP4 output, storage	Node.js + FFmpeg
Analytics Service	3104	Async metrics, reporting, aggregation	Node.js + Postgres
Presence Service	3105	Online status, Redis pub/sub	Express + Redis

3. TECHNOLOGY STACK DECISIONS

3.1 Why This Stack?

Node.js + TypeScript:

-  Single language (JavaScript) for all backend services
-  Native async/await handles 1000+ concurrent connections
-  Rich npm ecosystem (express, mediasoup, ffmpeg, pg)

-  Fast prototyping + type safety (TypeScript)

REST APIs (not gRPC):

-  Standard for Flutter client communication
-  Easier to debug (curl, Postman)
-  Simpler reverse proxy configuration (Nginx)
-  WebSockets for real-time signaling (not gRPC-web)

RabbitMQ (not NATS):

-  Durable queues (jobs survive server restart)
-  Dead-letter queues (failed jobs handled gracefully)
-  Message persistence (reliable job processing)
-  Standard in Cameroon hosting providers

PostgreSQL (not MongoDB):

-  ACID transactions (call state consistency)
-  Connection pooling support
-  Complex queries (analytics, reporting)
-  Foreign key constraints (data integrity)

Redis Cluster (not single node):

-  High availability (failover between nodes)
-  Pub/Sub for real-time presence updates
-  Session storage (JWT tokens, user cache)
-  Rate limiting (token bucket algorithm)

3.2 Cameroon-Specific Optimizations

Challenge: 3-10 Mbps average, 2-5% packet loss, high latency (100-200ms)

Solution 1: Adaptive Bitrate

Connection Start:

- Measure available bandwidth
- Default: 2 Mbps (safe for most Cameroon ISPs)

- Range: 0.5 Mbps (audio-only) → 8 Mbps (premium)
- Adjust every 2 seconds based on packet loss
- Fallback: Audio-only if video causes drops

Solution 2: Codec Selection

Primary: H.264 (better compression, lower CPU)

Fallback: VP8 (open-source, smaller bitrate)

Audio: Opus @ 48kHz (vs 64kHz) = 33% bandwidth savings

Solution 3: Forward Error Correction

- Enable FEC (duplicate packets every 10ms)
- Reduces impact of 3-5% packet loss
- Trade-off: +10% bandwidth overhead

Solution 4: NAT Traversal Priority

- STUN first (direct peer-to-peer, 70% success rate)
- TURN fallback (relay through Coturn, 30% of calls)
- Self-hosted Coturn in Cameroon (low latency)

Solution 5: Connection Management

- Connection pooling (reuse DB/Redis connections)
- Keep-alive on WebSockets (prevent timeout disconnections)
- Graceful degradation (continue on partial failures)
- Timeout: 30s for WebRTC negotiation (account for latency)

4. DETAILED SERVICE ARCHITECTURE

4.1 API Gateway (:3000)

Responsibilities:

- HTTP request validation
- JWT token verification
- Request logging + correlation ID injection
- Rate limiting (Redis token bucket)
- Response compression (Brotli)
- Route forwarding to internal services

Architecture:

Express App

├─ Middleware

| └─ tracingMiddleware (correlation ID)

| └─ requestLogger (structured logging)

| └─ rateLimiter (Redis-backed)

| └─ errorHandler (centralized)

| └─ compression (Brotli)

└─ Routes

├─ POST /api/v1/auth/login

├─ POST /api/v1/auth/register

├─ POST /api/v1/calls/create

├─ GET /api/v1/calls

├─ GET /api/v1/recordings

└─ GET /health

Key Features:

- Connection pooling to backend services
- Retry logic (3x with exponential backoff)
- Circuit breaker (fail fast on backend errors)
- Request timeout: 30s (account for Cameroon latency)

4.2 Auth Service (:3100)

Responsibilities:

- User registration
- Login (email/password)
- JWT token generation
- Token refresh
- Logout (token blacklist)

Database:

users table:

└ id (UUID)

└ email (unique)

└ phone (unique)

└ password_hash (bcrypt)

└ first_name, last_name

└ is_active (boolean)

└ created_at, updated_at

JWT Strategy:

- Access token: 15 minutes (short-lived)
- Refresh token: 7 days (stored in Redis)
- Algorithm: HS256
- Revocation: Remove refresh token from Redis on logout

Security:

- Rate limit: 5 failed logins → 15 min lockout
- Password: bcryptjs (12 rounds)
- HTTPS only (no HTTP)

4.3 User Service (:3101)

Responsibilities:

- User profile CRUD
- Contact list management
- User search
- Profile picture upload

Database:

users table: (as above)

contacts table:

└ id (UUID)

└ user_id (FK)

└ contact_id (FK)

└ created_at

└ deleted_at (soft delete)

API Endpoints:

GET /api/v1/users/:id

POST /api/v1/users/:id

DELETE /api/v1/users/:id

GET /api/v1/contacts

POST /api/v1/contacts

DELETE /api/v1/contacts/:id

GET /api/v1/users/search?q=name

4.4 Calls Service (:3102)

Responsibilities:

- Call creation + SFU assignment
- Call lifecycle management
- Participant tracking
- Quality monitoring
- SFU orchestration

Database:

calls table:

- └ id (UUID)
- └ initiator_id (FK users)
- └ call_type ('p2p' or 'group')
- └ room_id (unique, e.g., CALL_ABC123)
- └ sfu_instance_id
- └ start_time, end_time
- └ duration_seconds
- └ recording_id (FK)
- └ created_at

call_participants table:

- └ id (UUID)
- └ call_id (FK)
- └ user_id (FK)
- └ join_time, leave_time

└─ video_enabled, audio_enabled

└─ ip_address (for geo-location)

call_quality_metrics table:

└─ id (BIGSERIAL)

└─ call_id (FK)

└─ user_id (FK)

└─ timestamp

└─ video_bitrate_kbps

└─ audio_bitrate_kbps

└─ packet_loss_percent

└─ latency_ms

└─ jitter_ms

└─ frame_rate

SFU Selection Algorithm:

// Load balancing with affinity

```
function selectSFU(callId) {
```

```
    const callAffinity = redis.get(`call:affinity:${callId}`);
```

```
    if (callAffinity) return callAffinity; // Return existing SFU
```

```
    const instances = ['sfu-1', 'sfu-2'];
```

```
    const loads = instances.map(i => getInstanceLoad(i));
```

```
    const leastLoaded = instances[loads.indexOf(Math.min(...loads))];
```

```
redis.set(`call:affinity:${callId}`, leastLoaded, 24 * 60 * 60);

return leastLoaded;

}
```

API Endpoints:

POST /api/v1/calls/create

Request: { call_type: "p2p"|"group" }

Response: { call_id, room_id, sfu_instance, ice_servers: [...] }

GET /api/v1/calls?limit=20&offset=0

Response: { calls: [...], pagination: {...} }

POST /api/v1/calls/:id/end

Response: { call_id, duration_seconds, participants_count }

GET /api/v1/calls/:id/quality

Response: { metrics: { bitrate, packet_loss, latency, ... } }

4.5 Recording Service (:3103)

Responsibilities:

- Listen to RabbitMQ for recording jobs
- Capture RTP streams from Mediasoup
- Encode to MP4 using FFmpeg
- Store on disk
- Metadata tracking

Job Flow:

1. Call ends
2. Calls Service publishes: call.ended event

3. Recording Service consumes event
4. FFmpeg subprocess: RTP input → MP4 output
5. Post-processing: Move to permanent storage
6. Database: Update recording status + metadata
7. S3 (optional): Upload backup copy

Database:

recordings table:

- └ id (UUID)
- └ call_id (FK)
- └ file_path (/recordings/call_abc.mp4)
- └ file_size_bytes
- └ duration_seconds
- └ codec ('h264')
- └ bitrate_kbps
- └ status ('pending' | 'processing' | 'completed' | 'failed')
- └ created_at
- └ expires_at (NOW + 30 days)

FFmpeg Command:

```
ffmpeg -i rtp://127.0.0.1:5000 \  
-c:v libx264 -preset fast -crf 28 \  
-c:a libopus -b:a 128k \  
/recordings/${callId}.mp4
```

Retention Policy:

- Keep 30 days on disk
- Cron job at 2 AM: Delete expired recordings
- Manual delete via API for privacy

API Endpoints:

GET /api/v1/recordings?limit=20&call_id=...

Response: { recordings: [...], pagination: {...} }

GET /api/v1/recordings/:id/download

Response: Binary MP4 stream

DELETE /api/v1/recordings/:id

Response: 204 No Content

4.6 Analytics Service (:3104)

Responsibilities:

- Listen to RabbitMQ for events
- Aggregate metrics
- Generate reports
- Dashboard data

Event Subscriptions:

call.initiated → Track call creation time

call.ended → Calculate duration, success rate

participant.joined → Track concurrent participants

participant.left → Update call count

recording.completed → Recording metadata

user.login → Active user count

Metrics to Track:

- Calls per day (7-day, 30-day trends)
- Average call duration
- Call success rate (% completed successfully)
- Video enabled % vs audio-only %
- Peak concurrent participants
- Average latency per call
- Packet loss distribution

Database:

daily_stats table:

└─ date (DATE)

└─ total_calls (INTEGER)

└─ avg_duration_seconds (INTEGER)

└─ success_rate_percent (DECIMAL)

└─ peak_concurrent (INTEGER)

└─ avg_bitrate_kbps (INTEGER)

└─ avg_packet_loss_percent (DECIMAL)

API Endpoints:

GET /api/v1/analytics/overview?days=7

Response: { total_calls, avg_duration, success_rate, ... }

GET /api/v1/analytics/calls-per-day?days=30

Response: [{ date, count }, ...]

GET /api/v1/analytics/quality?call_id=...

Response: { bitrate_trend, packet_loss, latency, ... }

4.7 Presence Service (:3105)

Responsibilities:

- Track user online/offline status
- Pub/Sub for real-time updates
- Activity tracking

Implementation:

Redis Sorted Set: users:online

└─ Member: user_id

└─ Score: timestamp (current time)

└─ Auto-expire after 30 seconds (client must refresh)

Redis Pub/Sub Channels:

└─ presence:updates → Broadcast online/offline

└─ user:{id}:status → Individual user status changes

└─ calls:active → Active call updates

API Endpoints:

GET /api/v1/presence/:user_id

Response: { is_online, last_seen, current_call_id }

POST /api/v1/presence/update

Request: { status: "online"|"idle"|"offline" }

Response: { status: "updated" }

GET /api/v1/presence/online-count

Response: { online_count, total_users, percentage }

WebSocket Events:

Server → Client:

```
{  
  
  type: "user-online",  
  
  user_id: "...",  
  
  timestamp: "2026-09-01T10:00:00Z"  
  
}  
  
{  
  
  type: "user-offline",  
  
  user_id: "...",  
  
  timestamp: "2026-09-01T10:05:00Z"  
  
}
```

4.8 WebSocket Gateway (:3001)

Responsibilities:

- WebRTC signaling (offer/answer/ICE)
- Real-time call notifications
- Heartbeat + connection keep-alive

Message Types:

// Outgoing (Server → Client)

```
{  
  
  type: "offer",  
  
  call_id: "...",  
  
  sdp: "v=0\r\no=...",
```

```
    iceGathering: true
  }
  {
    type: "answer",
    call_id: "...",
    sdp: "v=0\r\no=..."
  }
  {
    type: "ice-candidate",
    call_id: "...",
    candidate: { candidate, sdpMLineIndex, ... }
  }
  {
    type: "participant-joined",
    call_id: "...",
    user: { id, name }
  }
  {
    type: "participant-left",
    call_id: "...",
    user_id: "..."
```

```
}
```

```
{
```

```
  type: "call-ended",
```

```
  call_id: "...",
```

```
  duration_seconds: 125,
```

```
  reason: "user-initiated"|"timeout"
```

```
}
```

```
{
```

```
  type: "error",
```

```
  error_code: "SFU_UNAVAILABLE",
```

```
  message: "Could not reach SFU instance"
```

```
}
```

```
// Incoming (Client → Server)
```

```
{
```

```
  type: "offer",
```

```
  call_id: "...",
```

```
  sdp: "..."
```

```
}
```

```
{
```

```
  type: "answer",
```

```
  call_id: "...",
```

```
  sdp: "..."
```

```

}

{
  type: "ice-candidate",
  call_id: "...",
  candidate: { ... }
}

{
  type: "participant-state",
  call_id: "...",
  video_enabled: true,
  audio_enabled: true
}

{
  type: "heartbeat"
}

```

5. DATA LAYER ARCHITECTURE

5.1 PostgreSQL Schema

-- Core Tables

CREATE TABLE users (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```
email VARCHAR(255) UNIQUE NOT NULL,  
  
phone VARCHAR(20) UNIQUE,  
  
password_hash VARCHAR(255) NOT NULL,  
  
first_name VARCHAR(100),  
  
last_name VARCHAR(100),  
  
avatar_url VARCHAR(500),  
  
is_active BOOLEAN DEFAULT true,  
  
created_at TIMESTAMP DEFAULT NOW(),  
  
updated_at TIMESTAMP DEFAULT NOW(),  
  
deleted_at TIMESTAMP NULL  
  
);  
  
CREATE INDEX idx_users_email ON users(email);  
  
CREATE INDEX idx_users_phone ON users(phone);  
  
CREATE TABLE calls (  
  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    initiator_id UUID REFERENCES users(id) NOT NULL,  
  
    call_type VARCHAR(20) DEFAULT 'p2p',  
  
    room_id VARCHAR(50) UNIQUE NOT NULL,  
  
    sfu_instance_id VARCHAR(50),  
  
    start_time TIMESTAMP NOT NULL,  
  
    end_time TIMESTAMP NULL,
```

```
duration_seconds INTEGER,

recording_id UUID,

created_at TIMESTAMP DEFAULT NOW()

);

CREATE INDEX idx_calls_initiator ON calls(initiator_id);

CREATE INDEX idx_calls_created ON calls(created_at DESC);

CREATE INDEX idx_calls_sfu ON calls(sfu_instance_id);

CREATE TABLE call_participants (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    call_id UUID REFERENCES calls(id) ON DELETE CASCADE NOT NULL,

    user_id UUID REFERENCES users(id) NOT NULL,

    join_time TIMESTAMP NOT NULL,

    leave_time TIMESTAMP NULL,

    video_enabled BOOLEAN DEFAULT true,

    audio_enabled BOOLEAN DEFAULT true,

    ip_address INET

);

CREATE INDEX idx_call_participants_call ON call_participants(call_id);

CREATE INDEX idx_call_participants_user ON call_participants(user_id);

CREATE TABLE recordings (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    call_id UUID REFERENCES calls(id) NOT NULL,
```

```
file_path VARCHAR(500) NOT NULL,  
  
file_size_bytes BIGINT,  
  
duration_seconds INTEGER,  
  
codec VARCHAR(20) DEFAULT 'h264',  
  
bitrate_kbps INTEGER,  
  
status VARCHAR(20) DEFAULT 'pending',  
  
created_at TIMESTAMP DEFAULT NOW(),  
  
expires_at TIMESTAMP NOT NULL  
  
);  
  
CREATE INDEX idx_recordings_call ON recordings(call_id);  
  
CREATE INDEX idx_recordings_expires ON recordings(expires_at);  
  
CREATE TABLE call_quality_metrics (  
  
    id BIGSERIAL PRIMARY KEY,  
  
    call_id UUID REFERENCES calls(id) NOT NULL,  
  
    user_id UUID REFERENCES users(id) NOT NULL,  
  
    timestamp TIMESTAMP NOT NULL,  
  
    video_bitrate_kbps INTEGER,  
  
    audio_bitrate_kbps INTEGER,  
  
    packet_loss_percent DECIMAL(5,2),  
  
    latency_ms INTEGER,  
  
    jitter_ms INTEGER,
```

```
    frame_rate INTEGER

);

CREATE INDEX idx_metrics_call ON call_quality_metrics(call_id, timestamp DESC);

-- Analytics Tables

CREATE TABLE daily_stats (

    date DATE PRIMARY KEY,

    total_calls INTEGER,

    avg_duration_seconds INTEGER,

    success_rate_percent DECIMAL(5,2),

    peak_concurrent INTEGER,

    avg_bitrate_kbps INTEGER,

    avg_packet_loss_percent DECIMAL(5,2)

);

-- Contacts Table

CREATE TABLE contacts (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    user_id UUID REFERENCES users(id) ON DELETE CASCADE NOT NULL,

    contact_id UUID REFERENCES users(id) NOT NULL,

    created_at TIMESTAMP DEFAULT NOW(),

    deleted_at TIMESTAMP NULL

);

CREATE INDEX idx_contacts_user ON contacts(user_id);
```

5.2 Redis Strategy

Key Structure:

session:{user_id} → User session data (TTL: 7 days)

user:{user_id}:profile → Cached profile (TTL: 1 hour)

call:{call_id}:state → Call state (TTL: 24 hours)

rate-limit:{user_id}:{endpoint} → Rate limiter (TTL: 60 sec)

call:affinity:{call_id} → SFU affinity mapping (TTL: 24 hours)

Pub/Sub Channels:

presence:updates → User online/offline events

call:{call_id}:events → Call-specific events

users:online → Sorted set of online users

Eviction Policy:

- **allkeys-lru** (evict least recently used)
- Max memory: 4GB

Persistence:

- AOF (Append-Only File) - durability
- RDB snapshots every 6 hours

5.3 RabbitMQ Configuration

Exchanges:

call.events (topic exchange)

|— call.initiated

|— call.ended

|— participant.joined

└ participant.left

recording.events (topic exchange)

└ recording.started

└ recording.completed

└ recording.failed

analytics.events (topic exchange)

└ * (wildcard)

Queues:

queue:recording-jobs

└ Binding: call.events → call.ended

└ Durability: Durable

└ Consumer: Recording Service

queue:analytics-jobs

└ Binding: *.events → *.*

└ Durability: Durable

└ Consumer: Analytics Service

queue:notification-jobs

└ Binding: call.events → *

└ Durability: Durable

└ Consumer: Notification Service (future)

Dead-Letter Queue:

dlq:failed-jobs

└─ Captures: Messages that failed 3x retries

└─ TTL: 7 days

└─ Alert: Admin notified

6. RESILIENCE & ERROR HANDLING

6.1 Circuit Breaker Pattern

```
class CircuitBreaker {
```

```
  state: 'CLOSED' | 'OPEN' | 'HALF_OPEN' = 'CLOSED';
```

```
  failures = 0;
```

```
  threshold = 5;
```

```
  async call(fn) {
```

```
    if (this.state === 'OPEN') {
```

```
      if (Date.now() - lastFailureTime > 60000) {
```

```
        this.state = 'HALF_OPEN';
```

```
      } else {
```

```
        throw new Error('Circuit is OPEN');
```

```
      }
```

```
    }
```

```

try {

  const result = await fn();

  this.onSuccess();

  return result;

} catch (err) {

  this.onFailure();

  throw err;

}

}

}

```

Usage:

- Protect calls to SFU (if unreachable, fail fast)
- Protect calls to PostgreSQL (if slow, circuit opens)
- Protect calls to Redis (if down, fallback to in-memory cache)

6.2 Retry Logic with Exponential Backoff

```

async function retryWithBackoff(fn, maxRetries = 3) {

  for (let i = 0; i < maxRetries; i++) {

    try {

      return await fn();

    } catch (err) {

      const delay = Math.min(1000 * Math.pow(2, i), 10000);

      if (i === maxRetries - 1) throw err;

      await new Promise(resolve => setTimeout(resolve, delay));
    }
  }
}

```

```
}  
  
}  
  
}
```

6.3 Connection Pooling

PostgreSQL:

Max connections: 100

Min connections: 10 (keep warm)

Idle timeout: 30 seconds

Statement timeout: 30 seconds

Connection timeout: 5 seconds

Redis:

Max connections: 500

Retry strategy: Exponential backoff

Enable offline queue: true

RabbitMQ:

Connection pooling: 10 connections

Prefetch: 1 (one job at a time)

Message TTL: 7 days

Durable queues: true

7. DEPLOYMENT ARCHITECTURE

7.1 Docker Compose Stack

version: '3.8'

services:

Entry Layer

nginx:

image: nginx:alpine

ports:

- "80:80"

- "443:443"

volumes:

- ./nginx.conf:/etc/nginx/nginx.conf:ro

- ./certs:/etc/nginx/certs:ro

depends_on:

- api-gateway

networks:

- callwave

Microservices

api-gateway:

build: ./services/api-gateway

ports:

- "3000:3000"

environment:

NODE_ENV: production

PORT: 3000

REDIS_URL: redis://redis:6379

depends_on:

- auth-service

- user-service

- calls-service

- recording-service

networks:

- callwave

websocket-gateway:

build: ./services/websocket-gateway

ports:

- "3001:3001"

environment:

NODE_ENV: production

PORT: 3001

networks:

- callwave

auth-service:

build: ./services/auth-service

ports:

- "3100:3100"

environment:

NODE_ENV: production

PORT: 3100

DATABASE_URL:

postgresql://callwave:\${POSTGRES_PASSWORD}@postgres:5432/callwave

REDIS_URL: redis://redis:6379

JWT_SECRET: \${JWT_SECRET}

depends_on:

- postgres

- redis

networks:

- callwave

user-service:

build: ./services/user-service

ports:

- "3101:3101"

environment:

NODE_ENV: production

PORT: 3101

DATABASE_URL:

postgresql://callwave:\${POSTGRES_PASSWORD}@postgres:5432/callwave

REDIS_URL: redis://redis:6379

depends_on:

- postgres

- redis

networks:

- callwave

calls-service:

build: ./services/calls-service

ports:

- "3102:3102"

environment:

NODE_ENV: production

PORT: 3102

DATABASE_URL:

postgresql://callwave:\${POSTGRES_PASSWORD}@postgres:5432/callwave

REDIS_URL: redis://redis:6379

RABBITMQ_URL: amqp://guest:guest@rabbitmq:5672

MEDIASOUP_INSTANCES: sfu-1:3000,sfu-2:3000

depends_on:

- postgres

- redis

- rabbitmq

networks:

- callwave

recording-service:

build: ./services/recording-service

environment:

NODE_ENV: production

DATABASE_URL:

postgresql://callwave:\${POSTGRES_PASSWORD}@postgres:5432/callwave

RABBITMQ_URL: amqp://guest:guest@rabbitmq:5672

RECORDINGS_PATH: /recordings

volumes:

- recordings:/recordings

depends_on:

- postgres

- rabbitmq

networks:

- callwave

analytics-service:

build: ./services/analytics-service

ports:

- "3104:3104"

environment:

NODE_ENV: production

PORT: 3104

DATABASE_URL:

postgresql://callwave:\${POSTGRES_PASSWORD}@postgres:5432/callwave

RABBITMQ_URL: amqp://guest:guest@rabbitmq:5672

depends_on:

- postgres

- rabbitmq

networks:

- callwave

presence-service:

build: ./services/presence-service

ports:

- "3105:3105"

environment:

NODE_ENV: production

PORT: 3105

REDIS_URL: redis://redis:6379

depends_on:

- redis

networks:

- callwave

Data Layer

postgres:

image: postgres:14-alpine

environment:

POSTGRES_DB: callwave

POSTGRES_USER: callwave

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD}

volumes:

- postgres-data:/var/lib/postgresql/data

- ./database/schema.sql:/docker-entrypoint-initdb.d/schema.sql

healthcheck:

test: ["CMD-SHELL", "pg_isready -U callwave"]

interval: 10s

timeout: 5s

retries: 5

networks:

- callwave

redis:

image: redis:7-alpine

volumes:

- redis-data:/data

healthcheck:

test: ["CMD", "redis-cli", "ping"]

interval: 10s

timeout: 5s

retries: 5

networks:

- callwave

rabbitmq:

image: rabbitmq:3.12-alpine

environment:

RABBITMQ_DEFAULT_USER: guest

RABBITMQ_DEFAULT_PASS: guest

volumes:

- rabbitmq-data:/var/lib/rabbitmq

healthcheck:

test: ["CMD", "rabbitmq-diagnostics", "ping"]

interval: 10s

timeout: 5s

retries: 5

networks:

- callwave

Mediasoup SFU

sfu-1:

build: ./mediasoup

environment:

INSTANCE_ID: sfu-1

REDIS_URL: redis://redis:6379

MEDIASOUP_LISTEN_IP: 0.0.0.0

MEDIASOUP_ANNOUNCED_IP: \${SERVER_IP}

MEDIASOUP_MIN_PORT: 40000

MEDIASOUP_MAX_PORT: 40100

ports:

- "40000-40100:40000-40100/udp"

depends_on:

- redis

networks:

- callwave

sfu-2:

build: ./mediasoup

environment:

INSTANCE_ID: sfu-2

REDIS_URL: redis://redis:6379

MEDIASOUP_LISTEN_IP: 0.0.0.0

MEDIASOUP_ANNOUNCED_IP: \${SERVER_IP}

MEDIASOUP_MIN_PORT: 40100

MEDIASOUP_MAX_PORT: 40200

ports:

- "40100-40200:40100-40200/udp"

depends_on:

- redis

networks:

- callwave

volumes:

postgres-data:

redis-data:

rabbitmq-data:

recordings:

networks:

callwave:

driver: bridge

7.2 Hardware Requirements

For 1,000 concurrent users:


```
"refresh_token": "refresh_...",  
  
"expires_in": 900,  
  
"user": {  
  
  "id": "uuid",  
  
  "email": "user@example.com",  
  
  "first_name": "John",  
  
  "last_name": "Doe"  
  
}  
}
```

POST /api/v1/auth/register

```
{  
  
  "email": "newuser@example.com",  
  
  "password": "password123",  
  
  "first_name": "Jane",  
  
  "last_name": "Doe"  
  
}
```

Response (201):

```
{  
  
  "user": { "id", "email", ... }  
  
}
```

POST /api/v1/auth/refresh

```
{  
  "refresh_token": "refresh_..."  
}
```

Response (200):

```
{  
  "access_token": "...",  
  "expires_in": 900  
}
```

POST /api/v1/auth/logout

Authorization: Bearer <TOKEN>

Response (204): No Content

8.2 Call Endpoints

POST /api/v1/calls/create

Authorization: Bearer <TOKEN>

```
{  
  "call_type": "p2p" | "group"  
}
```

Response (201):

```
{
```

```
"call_id": "uuid",

"room_id": "CALL_ABC123",

"sfu_instance": "sfu-1",

"ice_servers": [

  {

    "urls": ["stun:coturn.cameroon.local:3478"],

    "username": "videocall",

    "credential": "password"

  }

]

}
```

GET /api/v1/calls?limit=20&offset=0

Authorization: Bearer <TOKEN>

Response (200):

```
{

  "calls": [

    {

      "id": "uuid",

      "initiator_id": "uuid",

      "call_type": "p2p",

      "room_id": "CALL_ABC123",
```

```
    "start_time": "2026-09-01T10:30:00Z",  
    "end_time": "2026-09-01T10:45:30Z",  
    "duration_seconds": 930,  
    "recording_id": "uuid",  
    "participants_count": 2  
  }  
],  
  "pagination": { "total": 100, "limit": 20, "offset": 0 }  
}  
---
```

POST /api/v1/calls/:id/end

Authorization: Bearer <TOKEN>

Response (200):

```
{  
  "call_id": "uuid",  
  "call_state": "ended",  
  "duration_seconds": 930  
}
```

8.3 Recording Endpoints

GET /api/v1/recordings?limit=20&call_id=<uuid>

Authorization: Bearer <TOKEN>

Response (200):

```
{  
  "recordings": [  
    {  
      "id": "uuid",  
      "call_id": "uuid",  
      "file_path": "/recordings/call_abc.mp4",  
      "file_size_bytes": 52428800,  
      "duration_seconds": 930,  
      "codec": "h264",  
      "bitrate_kbps": 3000,  
      "created_at": "2026-09-01T10:45:30Z",  
      "expires_at": "2026-10-01T10:45:30Z",  
      "status": "completed"  
    }  
  ]  
}
```

GET /api/v1/recordings/:id/download

Authorization: Bearer <TOKEN>

Response: Binary MP4 stream

Content-Type: video/mp4

DELETE /api/v1/recordings/:id

Authorization: Bearer <TOKEN>

Response (204): No Content

9. SECURITY

9.1 Authentication & Authorization

Login Flow:

1. Client sends email + password (HTTPS POST)
2. Auth Service hashes password (bcryptjs, 12 rounds)
3. Compare with stored hash
4. Generate JWT tokens (access + refresh)
5. Store refresh token in Redis
6. Return tokens to client

Token Validation:

- Access token: HS256, 15 min TTL
- Refresh token: 7 days TTL
- Revocation: Remove from Redis on logout

Rate Limiting:

- 5 failed logins → 15 min lockout
- API requests: 1000/minute per user
- WebSocket connections: Max 100 per user

9.2 Data Encryption

In Transit:

- HTTPS (TLS 1.3) for all REST endpoints
- WSS (WebSocket Secure) for signaling
- DTLS-SRTP for media (enforced by Mediasoup)

At Rest:

- PostgreSQL: Native encryption (pgcrypto extension)
- Redis: AOF persistence (optional SSL)
- Recordings: MP4 files on disk (encrypted volume recommended)

9.3 Compliance

- **GDPR:** User data export + soft delete
 - **Privacy:** Recording consent required
 - **Data residency:** All data in Cameroon
 - **Audit trail:** Admin actions logged
-

10. MONITORING & OBSERVABILITY

10.1 Structured Logging

Format (JSON):

```
{  
  
  "timestamp": "2026-09-01T10:00:00Z",  
  
  "level": "INFO",  
  
  "service": "calls-service",  
  
  "correlationId": "abc-123-def",  
  
  "message": "Call initiated",  
  
  "data": {  
  
    "call_id": "uuid",  
  
    "initiator_id": "uuid",  
  
    "duration_ms": 45  
  
  }  
}
```

}

Log Levels:

- ERROR: Failures (auth, DB, SFU)
- WARN: Degradation (packet loss >5%, TURN fallback)
- INFO: Events (call start/end, login)
- DEBUG: Disabled in production

Log Destinations:

- stdout (Docker logs)
- Files (rotated daily, /logs)
- Elasticsearch (optional, for search)

10.2 Metrics (Prometheus)

Key Metrics:

http_requests_total{method, path, status}

http_request_duration_seconds{method, path, quantile}

db_queries_total{query_type, status}

db_query_duration_seconds{quantile}

rabbitmq_messages_total{queue, status}

jwt_validations_total{status}

call_duration_seconds{quantile}

sfu_participant_count{sfu_instance}

redis_hits_total

redis_misses_total

Scrape Interval: 30 seconds (not 15s, to reduce overhead)

10.3 Distributed Tracing (Jaeger - Optional)

Setup:

- Sample rate: 1:100 in production (10% in staging)
 - Trace: Request from gateway → service calls → DB queries
 - Jaeger UI: <http://localhost:16686>
-

11. BACKUP & DISASTER RECOVERY

11.1 Backup Strategy

PostgreSQL (Daily):

3:00 AM daily

```
pg_dump --host=localhost callwave | gzip > /backups/callwave_$(date +%Y%m%d).sql.gz
```

Keep 30 days

```
find /backups -type f -mtime +30 -delete
```

Recordings:

- Symlink to secondary disk (RAID-1 mirroring)
- Or: rsync to backup server nightly
- Auto-delete after 30 days (no manual action needed)

Redis:

- RDB snapshot every 6 hours (built-in)
- Used for recovery only (sessions recreated on restart)

11.2 Disaster Recovery

Scenario: Server crash

1. Restore PostgreSQL from latest backup (~1 hour)
2. Restart Docker Compose
3. Data loss: Active calls + last 30 min of recordings

Scenario: SFU crash

- Mediasoup auto-restart via Docker health check
- Participants rejoin call (30s reconnect window)
- No data loss

RTO (Recovery Time Objective): 1 hour **RPO (Recovery Point Objective):** 30 minutes

12. OPERATIONAL PROCEDURES

12.1 Deployment Checklist

1. Prepare server

```
sudo apt-get update && apt-get install -y docker.io docker-compose git
```

2. Clone repo

```
git clone https://github.com/yourorg/callwave.git
```

```
cd callwave
```

3. Setup environment

```
cp .env.example .env
```

```
# Edit: POSTGRES_PASSWORD, JWT_SECRET, SERVER_IP
```

4. Create volumes

```
mkdir -p /data/callwave/{recordings,postgres-data,redis-data,rabbitmq-data}
```

```
chmod 755 /data/callwave
```

5. Build images

```
docker-compose build
```

6. Start services

```
docker-compose up -d
```

```
# 7. Verify health
```

```
docker-compose ps
```

```
curl http://localhost:3000/health
```

```
curl http://localhost:3100/health
```

```
# 8. Initialize database
```

```
docker-compose exec postgres psql -U callwave -c "SELECT version();" 
```

12.2 Common Operations

```
# View logs
```

```
docker-compose logs -f <service>
```

```
# Restart service
```

```
docker-compose restart <service>
```

```
# Scale SFU
```

```
docker-compose up -d --scale sfu=3
```

```
# Backup database
```

```
docker-compose exec postgres pg_dump -U callwave callwave | gzip > backup.sql.gz
```

```
# Restore database
```

```
gunzip < backup.sql.gz | docker-compose exec -T postgres psql -U callwave callwave
```

```
# Monitor resources
```

```
docker stats
```

13. SCALING STRATEGY

Current (1,000 concurrent): Single Docker Compose server

Phase 2 (2,500 concurrent):

- Add 2nd Mediasoup instance (2x SFU capacity)
- Add PostgreSQL read replica
- Nginx load balancing between SFUs

Phase 3 (5,000+ concurrent):

- Migrate to Kubernetes (multi-node cluster)
 - Auto-scaling for Mediasoup pods
 - Distributed PostgreSQL (replication + sharding)
 - Add Prometheus + Grafana monitoring
-

14. ASSUMPTIONS & DEPENDENCIES

Assumptions:

- Cameroon average ISP: 3-10 Mbps
- Desktop clients (webcam + mic)
- Linux server with Docker support
- Stable internet (UPS recommended)

External Dependencies:

- Coturn (self-hosted STUN/TURN)
- SSL certificate (Let's Encrypt)
- Backup storage (NAS or cloud)

Not Included (Future Phases):

- Mobile app (Flutter)
 - Admin dashboard (React)
 - Video conferencing room (>2 participants)
 - Advanced analytics (ML-based)
-

15. PERFORMANCE TARGETS (CAMEROON NETWORK)

Metric	Target	Method
API latency (p50)	<100ms	Connection pooling, caching
API latency (p99)	<300ms	Rate limiting, circuit breaker
Call setup time	<3s	SFU pre-warmth, parallel negotiation
Recording start	<1s	Async job processing
Error rate	<0.1%	Retries, fallback codecs
Participant capacity	2000/cluster	2x SFU instances
Data durability	99.99%	Sync replicas, AOF persistence

Document Version: 2.0

Last Updated: September 1, 2026

Status: PRODUCTION-READY

Author: Callwave Engineering Team

Cameroon Optimization:  Complete